

基础软件开发技术实践

智能软件工程实验报告

请输入实验标题

姓名	刘明卓	院系	计算学部
学号	2023111648	任课教师	刘佳琨
指导教师	刘佳琨	实验地点	格物楼 213
实验时间	7.9		
实验得分		研讨得分	
报告得分		实验总分	

一、实验内容

1. 理解大语言模型在代码审查任务中的应用原理。
2. 掌握 Prompt Engineering 的基本思想与设计方法。
3. 理解不同代码上下文对大语言模型性能的影响。
4. 能够利用大语言模型完成 MergePrediction 任务。
5. 能够利用大语言模型自动生成代码审查意见。
6. 能够分析不同 Prompt 设计和上下文信息对代码审查结果的影响。

二、实验内容

1. 数据准备：读取实验三中的测试数据，并构建代码审查任务所需的数据输入。
2. 构建代码上下文：分别构建以下不同类型的上下文：
 - 仅包含 Diff；
 - Diff + Pull Request 描述；

- Diff + Commit Message;
- Diff + 等其他信息。

3. Prompt 设计：针对每种上下文分别设计：

- Zero-shot Prompt;
- Few-shot Prompt;
- Chain-of-Thought Prompt;
- Role-based Prompt。

4. 模型推理：调用大语言模型分别完成 Merge Prediction 和 Review Comment Generation，记录模型输出及推理时间。

5. 结果分析：统计不同上下文和 Prompt 设计下模型性能，包括但不限于 Accuracy、Precision、Recall、F1-score、BLEU、ROUGE、推理时间，并比较不同 Prompt 设计和上下文组合的优缺点。

三、实验结果

要求：将实验获得的结果进行描述，基本内容包括：

3.1 数据集概况

本实验从实验一采集的 GitHub PR 数据中，选取带有代码审查评论（review comment）的 PR 作为候选数据，共筛选出 801 条有效 PR。从中随机抽取 100 条作为实验数据，涉及的 GitHub 仓库分布如表 1 所示。真实标签中，44 条 PR 最终被 Merge，56 条 PR 未被 Merge（即被 Close/Reject），合并率约为 44%。

表 1: 实验数据集仓库分布

仓库名称	kubernetes/kubernetes	microsoft/vscode	pytorch/pytorch	langchain-ai/langchain
其他				
PR 数量	35	33	20	12
0				
占比	35%	33%	20%	12%
0%				

针对每条 PR，构建了 4 种不同信息量的代码上下文（仅 Diff、Diff+PR 描述、Diff+Commit Message、Diff+ 额外信息），并设计了 4 种不同策略的 Prompt（Zero-shot、Few-shot、Chain-of-Thought、Role-based），形成 $4 \times 4 = 16$ 组实验配置，每组配

置对 100 条 PR 进行推断，总计完成 1600 次 LLM 调用（每个任务方向）。实验中分别调用智谱 GLM-4.7-Flash 模型完成 Merge Prediction 和 Review Comment Generation 两个任务。

3.2 输出解析率分析

由于大语言模型的输出具有一定的不确定性，首先对模型输出的 JSON 格式解析率进行统计，结果如表 2 所示。

表 2: 不同 Prompt 类型的 JSON 输出解析率对比 (Merge Prediction 任务)						
Prompt 类型	总记录数	解析成功	解析率	Unknown	降级匹配 Yes	降级匹配 No
Zero-shot	400	345	86.2%	49	0	6
Few-shot	400	399	99.8%	0	0	1
Chain-of-Thought	400	301	75.2%	3	28	68
Role-based	400	300	75.0%	100	0	0

从解析率可以看出：

- Few-shot Prompt 的解析率最高，达到 99.8%，几乎零失败。这是因为 Few-shot 示例中展示了标准的 JSON 输出格式，模型能够严格遵循示例格式。
- Zero-shot Prompt 解析率为 86.2%，表现尚可，但有 12.2% 的记录完全无法解析为 Yes/No 决策。
- Chain-of-Thought 和 Role-based Prompt 的解析率最低，仅约 75%。Chain-of-Thought 由于要求模型进行多步推理，输出文本较长，容易在推理过程中偏离 JSON 格式；Role-based Prompt 由于赋予了模型”资深 Committer”角色，模型倾向于输出冗长的分析文字而非简洁的 JSON。

进一步按上下文类型拆分解析率，可以发现 ‘diff_extra’ Role-
basedPrompt 69%

3.3 Merge Prediction 整体结果

对解析成功的记录计算分类指标，与真实的 Merge/Not Merge 标签对比，整体结果如表 3 所示。

从整体结果可以观察到：

1. 模型在所有 Prompt 下 Recall 值都较高 (0.81~0.93)，说明模型倾向于预测”Yes”（即建议 Merge）。这与数据集的正负样本分布有关：由于大部分 PR 在开源项目中最终都会被 Merge，模型从预训练语料中学习到了”默认通过”的审查倾向。

表 3: Merge Prediction 各 Prompt 类型整体性能对比

Prompt 类型	有效样本数	Accuracy	Precision	Recall	F1-score	ROC-AUC
Zero-shot	351	0.5356	0.4792	0.9139	0.6287	0.5820
Few-shot	400	0.5100	0.4679	0.8295	0.5984	0.5442
Chain-of-Thought	397	0.4987	0.4610	0.8114	0.5880	0.5318
Role-based	300	0.5100	0.4567	0.9280	0.6121	0.5697

2. Zero-shot Prompt 在 Accuracy、F1-score 和 ROC-AUC 三个指标上均取得了最佳结果，这一结果反直觉——通常认为提供示例（Few-shot）或引导推理（Chain-of-Thought）会提升性能，但在本实验中简单直接的 Zero-shot Prompt 反而表现最好。可能的原因是 GLM-4.7-Flash 作为较新的模型已经具备较强的代码理解能力，额外的示例和推理引导反而引入了与目标任务不一致的偏见。
3. Four 种 Prompt 下 F1-score 的绝对差距并不大（0.588~0.629），说明 Prompt 策略对整体性能的影响主要体现在解析稳定性和推理效率上，而非核心预测能力。

3.4 上下文信息对性能的影响

将结果按上下文类型进行拆分，对比 4 种上下文在不同 Prompt 下的 F1-score 和 Accuracy，结果如表 4 和表 5 所示。

表 4: 不同上下文 × Prompt 组合的 F1-score 对比

Prompt 类型	diff_only	diff_pr_desc	diff_commit	diff_extra	平均值
Zero-shot	0.6261	0.6016	0.6066	0.7089	0.6358
Few-shot	0.5938	0.5942	0.5954	0.6154	0.5997
Chain-of-Thought	0.5645	0.5984	0.5455	0.6486	0.5893
Role-based	0.6139	0.6122	0.5941	0.6329	0.6133
各上下文平均值	0.5996	0.6016	0.5854	0.6515	—

从上下文维度分析，可以得出以下结论：

1. 额外信息上下文（diff_extra）显著最优

在所有 4 种 Prompt 策略下，‘diff_extra’ 的 F1-score 和 Accuracy 均显著优于其他策略。Zero-shot Prompt 的 Accuracy 为 0.7326，F1-score 为 0.7089。

这一结果表明：文件名、修改函数名等结构信息能够帮助模型定位代码变更的功能模块，而历史审查评论则提供了该代码领域的审查风格和关注点，两者结合为模型提供了超越纯代码差异的重要决策依据。

表 5: 不同上下文 × Prompt 组合的 Accuracy 对比

Prompt 类型	diff_only	diff_pr_desc	diff_commit	diff_extra	平均值
Zero-shot	0.5114	0.4494	0.4545	0.7326	0.5370
Few-shot	0.4800	0.4400	0.4700	0.6500	0.5100
Chain-of-Thought	0.4490	0.4900	0.4444	0.6100	0.4984
Role-based	0.5000	0.5000	0.4675	0.5797	0.5118
各上下文平均值	0.4851	0.4699	0.4591	0.6431	—

2. PR 描述与 Commit Message 的增益有限

对比 ‘diff_only’ Diff ‘diff_pr_desc’ Diff+PR ‘diff_commit’ Diff+CommitMessage P
shot Few-shot PR F1-score

可能的原因是：PR 描述和 Commit Message 通常偏向功能描述而非质量评估，包含的信息与代码审查任务的相关性较低，甚至可能引入噪声，分散模型对代码变更本身的关注。

3. Zero-shot + diff_extra 是最佳组合

综合 F1-score 和 Accuracy, Zero-shot Prompt + diff_extra 上下文是 16 种组合中的最佳配置：

- Accuracy: 0.7326（所有组合中最高）
- F1-score: 0.7089（所有组合中最高）
- ROC-AUC: 0.7389（所有组合中最高）
- 解析率: 84%（在可接受范围内）
- 平均延迟: 7.2s（仅次于 Few-shot 的 5.9s）

这一组合兼具了良好的预测性能和较高的推理效率，是本实验条件下的推荐配置。

3.5 推理延迟分析

不同 Prompt 策略的平均推理延迟如表 6 所示。

推理延迟与 Prompt 设计直接相关：Chain-of-Thought 要求模型输出分步推理过程，Role-based 引导模型扮演资深开发者角色进行深度分析，两者均显著增加了输出 token 数量，从而导致推理时间大幅增加。从工程实践角度，Few-shot 和 Zero-shot 在推理效率上具有明显优势。

表 6: 不同 Prompt 类型的平均推理延迟

Prompt 类型	平均延迟	相对 Few-shot 倍数
Few-shot	5.9s	1.0x
Zero-shot	7.2s	1.2x
Chain-of-Thought	17.9s	3.0x
Role-based	37.3s	6.3x

3.6 Review Comment Generation 结果

针对 Review Comment Generation 任务，使用 BLEU 和 ROUGE-L 指标对模型生成的审查评论与原始审查评论进行对比。由于审查评论的语言具有较强的主观性——不同开发者对同一代码变更可能给出完全不同但同样合理的审查意见——基于 n-gram 匹配的 BLEU 和 ROUGE 指标在此任务上的绝对数值较低，这符合自然语言生成任务的典型特征。

在 Zero-shot + diff_extra 配置下，生成的审查评论能够较为准确地指出代码变更中的潜在问题，部分输出示例体现了模型对代码语义的理解能力。相较而言，Chain-of-Thought Prompt 生成的评论文本更长、分析维度更多，但也更容易输出与原始评论风格不一致的内容。

3.7 与实验三深度学习模型的对比

将本实验中最优 LLM 配置 (Zero-shot + diff_extra, F1=0.7089) 与实验三的 CodeBERT 深度学习模型进行横向对比，可以发现：

- LLM 无需针对特定任务进行模型训练，只需设计合适的 Prompt 即可完成任务，工程成本显著降低；
- LLM 的输入格式灵活，能够直接处理自然语言描述与代码文本的混合输入，为引入更多元的上下文信息提供了便利；
- 但 LLM 的输出具有不确定性，解析率和性能稳定性是需要关注的工程问题；
- 在本实验条件下，LLM 的 F1-score (0.7089) 与专门训练的 CodeBERT 模型相比仍有一定差距，说明针对特定代码审查任务的监督学习模型在性能上仍具优势，但 LLM 的通用性和灵活性使其在实际应用场景中具有独特价值。

四、实验中遇到的问题总结

要求：主要阐述实验过程中遇到的问题如何解决的。

问题 1: api 花费的时间有点多

解决 1: 晚上开着, 可以用并行发丝部分

五、实验体会

大模型相较于传统的方法效果比较好还比较方便

指导教师评语:

日期: